Universidad Pontificia de Salamanca
Facultad de Informática

Práctica de Informática Teórica

Análisis léxico con Lex/Flex — lenguaje **miniLang**

Curso 2026-2027

1 Objetivo de la práctica

El objetivo de esta práctica es comprender el funcionamiento del **analizador léxico**, primera fase de un compilador, y adquirir soltura en el uso de **Lex/Flex** para reconocer los componentes léxicos de un lenguaje de programación.

Se deberá construir un analizador capaz de leer un programa escrito en el lenguaje **miniLang**, identificar sus tokens, contabilizar las líneas del programa y detectar errores léxicos.

2 Descripción léxica del lenguaje miniLang

`miniLang` es un lenguaje reducido que se utilizará también en la práctica posterior de análisis sintáctico y semántico con Yacc.

2.1 Palabras reservadas

Las únicas palabras reservadas del lenguaje son:

```
program
var
const
int
real
bool
true
false
if
else
for
to
downto
print
read
and
or
not
```

2.2 Operadores

Se reconocerán los siguientes operadores:

```
=       asignación

+       suma
-       resta o cambio de signo
*       multiplicación
/       división
%       resto de la división entera
**      potencia

<       menor que
>       mayor que
<=      menor o igual que
>=      mayor o igual que
==      igual que
!=      distinto de

and     conjunción lógica
or      disyunción lógica
not     negación lógica
```

2.3 Separadores

Los separadores del lenguaje son:

```
;   :   (   )   {   }
```

2.4 Identificadores

Un identificador estará formado por letras, dígitos y el carácter _.

- Puede comenzar por una letra o por _.
- No puede coincidir con una palabra reservada.

Ejemplos válidos:

```
contador
_radio
precio2
resultado_final
```

2.5 Constantes numéricas

Se reconocerán dos tipos de constantes numéricas:

Constantes enteras

Ejemplos:

```
0
25
1024
```

Constantes reales

Podrán escribirse en notación decimal o científica.

Ejemplos:

```
3.14
0.5
12.0
1.3e-2
2E6
3.0E+8
```

Una constante en notación científica se considerará de tipo real aunque no contenga punto decimal.

2.6 Constantes de tipo cadena

Las cadenas se escribirán entre comillas dobles:

```
"Hola"
"Introduce un valor: "
"Resultado final"
```

Una cadena no podrá ocupar más de una línea.

2.7 Constantes booleanas

Las constantes booleanas serán exclusivamente:

```
true
false
```

2.8 Comentarios

El lenguaje admite dos tipos de comentarios.

Comentario de una línea

Comienza con `//` y termina al finalizar la línea.

```
// Esto es un comentario
```

Comentario de varias líneas

Comienza con `/*` y termina con `*/`

```
/* Este comentario
   ocupa varias líneas */
```

2.9 Espacios en blanco

Los espacios, tabuladores y saltos de línea se utilizarán como separadores. Los saltos de línea deberán contabilizarse para conocer el número total de líneas del fichero y para poder informar correctamente de los errores.

3 Trabajo que se debe realizar

Se pide implementar, usando **Lex/Flex**, un analizador léxico que procese un fichero fuente escrito en `miniLang`.

El programa deberá generar un fichero denominado:

```
tokens.txt
```

que contenga, en el orden en que aparecen en el programa, todos los tokens reconocidos.

3.1 Información asociada a los tokens

Para los identificadores y las constantes se mostrará también su lexema.

Ejemplos de salida:

```
PROGRAM
ID(ejemplo)
{
CONST
ID(PI)
=
CTE_REAL(3.1416)
;
VAR
ID(radio)
:
REAL
;
ID(radio)
=
-
CTE_REAL(1.3e-2)
;
PRINT
(
CTE_CADENA("Resultado")
)
;
}
```

3.2 Número de líneas

Al final del fichero `tokens.txt` deberá aparecer:

```
Número de líneas: N
```

donde `N` es el número total de líneas del fichero fuente.

3.3 Errores léxicos

El analizador deberá detectar e informar de algún error léxico.

Entre otros, pueden considerarse:

- carácter no reconocido;
- identificador no válido;
- constante numérica mal formada;
- cadena sin cerrar;
- comentario de varias líneas sin cerrar.

El mensaje deberá indicar, como mínimo, el número de línea y una descripción del error.

Ejemplo:

```
LÍNEA 8. ERROR LÉXICO: cadena sin cerrar
```

4 Ejemplo de programa escrito en miniLang

El siguiente programa ilustra la mayoría de los elementos léxicos que deben reconocerse:

```
program ejemplo {
    const PI = 3.1416;
    const ACTIVO = true;

    var radio:real;
    var altura:real = 10.0;
    var valido:bool = true;

    // Lectura del radio
    print("Introduce el radio: ");
    read(radio);

    if (radio > 0 and valido) {
        varResultado = 2 * PI * radio;
        print(varResultado);
    }

    /* Fin del
       cálculo */
}
```

5 Ficheros de prueba

Además del ejemplo anterior, cada grupo deberá preparar varios ficheros de entrada que permitan comprobar:

1. reconocimiento de todos los tipos de token
2. números enteros, reales y en notación científica
3. comentarios de una y varias líneas
4. constantes cadena
5. operadores de uno y dos caracteres
6. algún error léxico

Se recomienda que los ficheros fuente utilicen la extensión:

```
.min
```

6 Documentación y entrega

Se entregará un fichero comprimido que contendrá, como mínimo:

1. el fichero fuente de Lex/Flex (`.l`)
2. los ficheros de entrada utilizados en las pruebas (`.min`)
3. los ficheros de salida generados (`tokens.txt` u otros equivalentes)
4. un documento PDF con la memoria de la práctica.

La memoria deberá incluir:

- nombres y apellidos de los integrantes del grupo
- explicación de las expresiones regulares utilizadas
- explicación del tratamiento de comentarios
- explicación del control del número de línea
- errores léxicos detectados
- información sobre uso de herramientas de inteligencia artificial

El grupo podrá estar formado por un máximo de **4 personas**.

7 Uso de herramientas de inteligencia artificial

Si se ha utilizado alguna herramienta de inteligencia artificial como apoyo en la realización de la práctica, deberá indicarse en la memoria:

- qué herramienta se ha utilizado
- para qué se ha utilizado
- qué partes del trabajo se han generado o modificado con su ayuda (especificar todas)
- cómo se ha comprobado que la solución entregada es correcta y comprendida por los integrantes del grupo

Si no se ha utilizado ninguna herramienta de inteligencia artificial, deberá indicarse también expresamente.

8 Evaluación

Para la evaluación se tendrá en cuenta:

- corrección del analizador léxico y cobertura de todos los tokens especificados
- tratamiento de comentarios, cadenas y líneas
- detección de errores léxicos
- calidad y claridad del código
- calidad de la documentación y de los casos de prueba

Causas de suspenso

La práctica se calificará como suspensa, con independencia del resto de la evaluación, si se da alguna de estas circunstancias:

- No se cumplen los requisitos obligatorios del enunciado.
- El código, la memoria o las pruebas no tienen la calidad mínima exigible.
- Se ha utilizado inteligencia artificial sin declararlo, o los integrantes no saben explicar el trabajo entregado.
- Existen prácticas iguales o sustancialmente similares. En ese caso se suspenderán todas las prácticas afectadas.

El profesor podrá convocar una defensa oral para comprobar la autoría y la comprensión del trabajo. Las prácticas suspensas deberán realizar la práctica de la convocatoria extraordinaria.